

Estrutura do Repositório Pilha POO

<https://gemini.google.com/app/48ee9e93edb18a92>

User prompt: Classe Pilha: Esta tarefa possui a seguinte dinâmica: Serve como NOTA DE AVALIAÇÃO. Para ser feita em grupo Os resultados desta TAREFA serão alvo de avaliação em sala de aula, através de desafios de alteração. A discussão dos resultados poderá se estender através da utilização do Fórum do AVA@UFRJ. Esta participação pode render pontos extras na média final É permitido e encorajado o uso de assistentes de IA gerativa na execução desta tarefa, (como GitHub Copilot, ChatGPT, Claude ou assistentes integrados à IDE via extensão Continue.dev utilizando modelos gratuitos da Groq, Together AI ou Google Gemini API). As ferramentas podem ser utilizadas para geração de código inicial, apoio ao debug, sugestão de refatoração, documentação inline e escrita das baterias de testes. Contudo, o aluno é 100% responsável pela exatidão, desempenho e autoria das decisões de projeto. Caso utilize IA, certifique-se de configurar a integração com seu repositório no GitHub para manter o histórico de commits rastreável e registrar o processo de desenvolvimento. Utilizando o seu ambiente de desenvolvimento com geração de código com auxílio de IA, desenvolver o projeto de uma classe POO que implementa uma estrutura de dados Pilha que deve aceitar apenas dados de um mesmo tipo básico da linguagem de implementação escolhida (caractere, inteiro ou ponto flutuante). O armazenamento interno da pilha poderá ser inicialmente um array da biblioteca padrão da linguagem. Assim, caso se esteja usando C/C++, deverá ser um membro dado do tipo básico array em memória. No caso de se estar usando Python, deve ser uma instância da classe "array", da biblioteca padrão do Python (array — Efficient arrays of numeric values — Python 3.11.5 documentation). A especificação das funções (métodos) da interface dessa classe Pilha é a seguinte: empilha(dado) : empilha um dado no topo da pilha. Se a pilha estiver cheia, deve levantar a exceção "PilhaCheiaErro". Se o dado não for do tipo básico armazenado pela Pilha, deve levantar a exceção "TipoErro". Desempilha() -> dado : desempilha o dado do topo da pilha, retornando-o. Se a pilha estiver vazia deve levantar a exceção "PilhaVaziaErro". pilha_esta_vazia() : retorna True se a pilha estiver vazia . pilha_esta_cheia() : retorna True se a pilha estiver cheia . troca() : troca o dado do topo da pilha com o dado imediatamente abaixo . tamanho() : retorna o número de dados empilhados As duas últimas rotinas são utilizadas em certos tipos de aplicação de uma Pilha. O grupo deverá gerar três codificações dessas classe. Uma em Python, uma em C/C++ e outra em JavaScript. A resposta a esta tarefa é composta dos seguintes produtos e ações: Um link para o repositório do projeto (GitHub Classroom / GitLab CI) Relatório com os log dos prompts utilizados e o código bruto gerado pela IA Relatório de Code Review sobre o código da IA: ele deve identificar onde a IA gerou um código ineficiente Relatório de bateria de testes efetuados buscando problemas Relatório de Análise Crítica e Post-Mortem de IA: Além da submissão do código-fonte e dos testes no repositório, o trabalho deve ser acompanhado do arquivo POST_MORTEM.md na raiz do projeto. Este documento é o principal critério de avaliação da autoria e deve conter: Log de Iteração e Prompts: Apresente os prompts chave utilizados para gerar trechos relevantes da solução e os fragmentos de código brutos devolvidos pela IA antes de qualquer modificação manual. Code Review Crítico: Analise a solução inicial da IA identificando falhas de eficiência, complexidade de algoritmo, vazamentos de memória, acesso fora da faixa etc. Justificativa de Refatoração: Explique detalhadamente quais alterações você precisou realizar no código gerado para atender aos requisitos de desempenho, correção e testes de estresse solicitados. Evidência de Testes: Relatório simples dos testes de estresse executados, demonstrando que o código refatorado por você supera o código ingênuo gerado inicialmente pelo modelo. Os Relatórios em PDF devem conter uma FOLHA DE ROSTO com a lista dos nomes completos e DRE dos integrantes do grupo. Estes produtos devem ser postados nesta tarefa por APENAS UM componente do grupo. Os demais postam apenas a folha de rosto em PDF. O aluno que não postar nada, mesmo estando no grupo, terá a sua nota reduzida, podendo ser ZERO. Devido a possível problema de disponibilidade do AVA@UFRJ no dia do prazo final de entrega, será considerado com entregue o trabalho (PDF e SLIDES) enviado dentro do prazo para o endereço de correio eletrônico milton@matematica.ufrj.br A resposta a esta tarefa é composta dos seguintes produtos e ações: Um link para o repositório do projeto (GitHub Classroom / GitLab CI) Relatório com os log dos prompts utilizados e o código bruto gerado pela IA Relatório de Code Review sobre o código da IA: ele deve identificar onde a IA gerou um código ineficiente Relatório de bateria de testes efetuados buscando problemas Relatório de Análise Crítica e Post-Mortem de IA: Além da submissão do código-fonte e dos testes no repositório, o trabalho deve ser acompanhado do arquivo POST_MORTEM.md na raiz do projeto. Este documento é o principal critério de avaliação da autoria e deve conter: Log de Iteração e Prompts: Apresente os prompts chave utilizados para gerar trechos relevantes da solução e os fragmentos de código brutos devolvidos pela IA antes de qualquer modificação manual. Code Review Crítico: Analise a solução inicial da IA identificando falhas de eficiência, complexidade de algoritmo, vazamentos de memória, falta de índices ou problemas de concorrência. Justificativa de Refatoração: Explique detalhadamente quais alterações você precisou realizar no código gerado para atender aos requisitos de desempenho, correção e testes de estresse solicitados. Evidência de Testes: Relatório simples dos testes de estresse executados, demonstrando que o código refatorado por você supera o código ingênuo gerado inicialmente pelo modelo. Os Relatórios em PDF devem conter uma FOLHA DE ROSTO com a lista dos nomes completos e DRE dos integrantes do grupo. Estes produtos devem ser postados nesta tarefa por APENAS UM componente do grupo. Os demais postam apenas a folha de rosto em PDF. O aluno que não postar nada, mesmo estando no grupo, terá a sua nota reduzida, podendo ser ZERO. Devido a possível problema de disponibilidade do AVA@UFRJ no dia do prazo final de entrega, será considerado com entregue o trabalho (PDF e SLIDES) enviado dentro do prazo para o endereço de correio eletrônico milton@matematica.ufrj.br Como será a estrutura do repositório?

Response: Para organizar um repositório profissional, rastreável e alinhado aos critérios de avaliação da UFRJ, a estrutura deve ser modular, separando as três linguagens, os testes e a documentação requerida.

```
meu-projeto-pilha/
├── .github/
│   ├── workflows/
│   └── tests.yml
│   # Configurações do repositório
│   # (Opcional) CI/CD com ações automatizadas
├── doc/
│   └── Relatorio_Final_Grupo.pdf
│   # Relatórios e artefatos em PDF para entrega
│   # Contém a Folha de Rosto (Nomes/DREs) e os relatórios
├── python/
│   ├── pilha.py
│   ├── excecoes.py
│   └── test_pilha.py
│   # Implementação em Python
│   # Classe Pilha (usando a biblioteca padrão 'array')
│   # Exceções customizadas (PilhaCheiaErro, TipoErro, etc.)
│   # Bateria de testes e teste de estresse
├── cpp/
│   ├── Pilha.hpp
│   ├── Pilha.cpp
│   ├── Excecoes.hpp
│   └── main_test.cpp
│   # Implementação em C/C++
│   # Declaração da Classe (Template/Array estático em memória)
│   # Implementação dos métodos
│   # Exceções customizadas
│   # Bateria de testes e testes de estresse
└── js/
    # Implementação em JavaScript (Node.js)
```

├── Pilha.js	# Classe Pilha (usando TypedArrays como Float64Array, Int32Array)
├── Excecoes.js	# Exceções customizadas
├── pilha.test.js	# Bateria de testes
├── .gitignore	# Arquivos a serem ignorados (compilados, cache, etc.)
├── README.md	# Visão geral do projeto e instruções de execução
└── POST_MORTEM.md	# EXIGÊNCIA OBRIGATÓRIA DA AVALIAÇÃO

Detalhamento dos Arquivos Principais

- **POST_MORTEM.md (Raiz do projeto):**
 - **Log de Prompts:** Registro exato das instruções enviadas para a IA e o código bruto retornado.
 - **Code Review Crítico:** Análise de falhas no código gerado (ex: uso ineficiente de `list` em Python ao invés da biblioteca `array`, falta de gerenciamento de memória em C++, complexidade das trocas).
 - **Justificativa de Refatoração:** Alterações feitas para ajustar exceções, limites e desempenho.
 - **Evidência de Testes:** Gráficos ou logs comparando o desempenho/tempo de execução do código original da IA *versus* o refatorado.
- **Armazenamento Interno Exigido em Cada Linguagem:**
 - **Python:** Deve utilizar `import array` (como `array.array('i')` para inteiros).
 - **C/C++:** Deve utilizar `array` básico/membro em memória (como `T dados[CAPACIDADE]` ou ponteiro dinâmico `T* dados`).
 - **JavaScript:** Deve utilizar `TypedArrays` (como `Int32Array`, `Float64Array` ou `Uint8Array`) para garantir o requisito de **tipo básico fixo**.
- **doc/Relatorio_Final_Grupo.pdf:**
 - Contém a **Folha de Rosto** obrigatória (Nomes completos e DREs de todos os integrantes).

User prompt: Gere o código inicial da classe Pilha com as exceções personalizadas em Python, C++ e JavaScript.

Response: Abaixo está a implementação inicial completa da classe **Pilha** com suas exceções personalizadas em **Python**, **C++** e **JavaScript**, atendendo rigorosamente a todas as especificações da tarefa (tipagem estrita/básica, armazenamento em `array` e interface de métodos requisitada).

1. Implementação em Python (python/)

Em Python, utiliza-se a classe `array.array` da biblioteca padrão para garantir que apenas valores do mesmo tipo numérico/básico sejam armazenados em memória contígua.

excecoes.py

```
class PilhaCheiaErro(Exception):
    """Levantada ao tentar empilhar em uma pilha que atingiu a capacidade máxima."""
    pass

class PilhaVaziaErro(Exception):
    """Levantada ao tentar desempilhar ou acessar elementos de uma pilha vazia."""
    pass

class TipoErro(TypeError):
    """Levantada ao tentar empilhar um dado de tipo incompatível com a pilha."""
    pass
```

pilha.py

```
import array
from excecoes import PilhaCheiaErro, PilhaVaziaErro, TipoErro

class Pilha:
    """
    Implementação de Pilha sobre a biblioteca padrão 'array' de Python.

    Códigos de tipo ('typecode'):
    - 'i': inteiro assinado (int)
    - 'd': ponto flutuante de dupla precisão (float)
    - 'u': caractere unicode (char)
    """
```

```

def __init__(self, capacidade: int, typecode: str = 'i'):
    if capacidade <= 0:
        raise ValueError("A capacidade deve ser um número inteiro positivo.")

    self._capacidade = capacidade
    self._typecode = typecode
    self._dados = array.array(typecode)

def empilha(self, dado):
    if self.pilha_esta_cheia():
        raise PilhaCheiaErro("A pilha está cheia. Não é possível empilhar mais elementos.")

    try:
        self._dados.append(dado)
    except (TypeError, ValueError) as err:
        raise TipoErro(f"0 dado '{dado}' não é compatível com o tipo estipulado '{self._typecode}'") from err

def desempilha():
    """Desempilha o dado do topo e o retorna."""
    if self.pilha_esta_vazia():
        raise PilhaVaziaErro("A pilha está vazia. Não é possível desempilhar.")
    return self._dados.pop()

# Ajustado de acordo com a interface solicitada:
def desempilha(self):
    if self.pilha_esta_vazia():
        raise PilhaVaziaErro("A pilha está vazia. Não é possível desempilhar.")
    return self._dados.pop()

def pilha_esta_vazia(self) -> bool:
    return len(self._dados) == 0

def pilha_esta_cheia(self) -> bool:
    return len(self._dados) >= self._capacidade

def troca(self):
    """Troca o elemento do topo com o elemento imediatamente abaixo."""
    if len(self._dados) < 2:
        raise PilhaVaziaErro("A pilha necessita de pelo menos dois elementos para efetuar a troca.")

    self._dados[-1], self._dados[-2] = self._dados[-2], self._dados[-1]

def tamanho(self) -> int:
    return len(self._dados)

```

2. Implementação em C++ (cpp/)

Em C++, a classe é implementada utilizando *Templates* e gerenciamento de array estático em memória alocada dinamicamente (T* dados), atendendo ao requisito de armazenamento interno em array básico.

Excecoes.hpp

```

#ifndef EXCECOES_HPP
#define EXCECOES_HPP

#include <stdexcept>
#include <string>

class PilhaCheiaErro : public std::runtime_error {
public:
    explicit PilhaCheiaErro(const std::string& msg = "Pilha Cheia")
        : std::runtime_error(msg) {}
};

class PilhaVaziaErro : public std::runtime_error {
public:
    explicit PilhaVaziaErro(const std::string& msg = "Pilha Vazia")
        : std::runtime_error(msg) {}
};

class TipoErro : public std::invalid_argument {
public:
    explicit TipoErro(const std::string& msg = "Tipo Incompativel")
        : std::invalid_argument(msg) {}
};

```

```

};

#endif

Pilha.hpp

#ifndef PILHA_HPP
#define PILHA_HPP

#include <cstdint>
#include "Excecoes.hpp"

template <typename T>
class Pilha {
private:
    T* dados;
    std::size_t capacidade;
    std::size_t topo; // Indica a quantidade de elementos atual

public:
    explicit Pilha(std::size_t cap) : capacidade(cap), topo(0) {
        if (cap == 0) {
            throw std::invalid_argument("A capacidade deve ser maior que zero.");
        }
        dados = new T[capacidade];
    }

    ~Pilha() {
        delete[] dados;
    }

    // Desabilita cópia rasa para evitar vazamento / double free
    Pilha(const Pilha&) = delete;
    Pilha& operator=(const Pilha&) = delete;

    void empilha(T dado) {
        if (pilha_esta_cheia()) {
            throw PilhaCheiaErro("A pilha atingiu a capacidade máxima.");
        }
        dados[topo++] = dado;
    }

    T desempilha() {
        if (pilha_esta_vazia()) {
            throw PilhaVaziaErro("A pilha está vazia.");
        }
        return dados[--topo];
    }

    bool pilha_esta_vazia() const {
        return topo == 0;
    }

    bool pilha_esta_cheia() const {
        return topo == capacidade;
    }

    void troca() {
        if (topo < 2) {
            throw PilhaVaziaErro("Pilha necessita de ao menos dois elementos para efetuar troca.");
        }
        T temp = dados[topo - 1];
        dados[topo - 1] = dados[topo - 2];
        dados[topo - 2] = temp;
    }

    std::size_t tamanho() const {
        return topo;
    }
};

#endif

```

3. Implementação em JavaScript (js/)

Em JavaScript, utiliza-se a família dos **TypedArrays** (Int32Array, Float64Array, Uint8Array, etc.) para garantir alocação contígua de memória e validação estrita de tipos básicos numéricos.

Excecoes.js

```
class PilhaCheiaErro extends Error {
  constructor(message = "Pilha Cheia") {
    super(message);
    this.name = "PilhaCheiaErro";
  }
}

class PilhaVaziaErro extends Error {
  constructor(message = "Pilha Vazia") {
    super(message);
    this.name = "PilhaVaziaErro";
  }
}

class TipoErro extends TypeError {
  constructor(message = "Tipo Incompatível") {
    super(message);
    this.name = "TipoErro";
  }
}

module.exports = { PilhaCheiaErro, PilhaVaziaErro, TipoErro };
```

Pilha.js

```
const { PilhaCheiaErro, PilhaVaziaErro, TipoErro } = require('./Excecoes');

class Pilha {
  /**
   * @param {number} capacidade - Capacidade máxima da pilha.
   * @param {Function} TypedArrayConstr - Construtor de TypedArray (ex: Int32Array, Float64Array).
   */
  constructor(capacidade, TypedArrayConstr = Int32Array) {
    if (capacidade <= 0) {
      throw new Error("Capacidade deve ser maior que zero.");
    }
    this._capacidade = capacidade;
    this._dados = new TypedArrayConstr(capacidade);
    this._topo = 0; // Quantidade atual de elementos
  }

  empilha(dado) {
    if (this.pilha_esta_cheia()) {
      throw new PilhaCheiaErro("A pilha atingiu a sua capacidade máxima.");
    }

    if (typeof dado !== 'number' || Number.isNaN(dado)) {
      throw new TipoErro(`Dado inválido ou tipo incompatível: ${dado}`);
    }

    this._dados[this._topo] = dado;
    this._topo++;
  }

  desempilha() {
    if (this.pilha_esta_vazia()) {
      throw new PilhaVaziaErro("A pilha está vazia.");
    }

    this._topo--;
    const elemento = this._dados[this._topo];
    return elemento;
  }

  pilha_esta_vazia() {
    return this._topo === 0;
  }
}
```

```

    pilha_esta_cheia() {
        return this._topo === this._capacidade;
    }

    troca() {
        if (this._topo < 2) {
            throw new PilhaVaziaErro("Pilha precisa de pelo menos 2 elementos para efetuar a troca.");
        }

        const temp = this._dados[this._topo - 1];
        this._dados[this._topo - 1] = this._dados[this._topo - 2];
        this._dados[this._topo - 2] = temp;
    }

    tamanho() {
        return this._topo;
    }
}

module.exports = Pilha;

```

Resumo dos Destaques de Projeto para o POST_MORTEM.md

1. **Python:** O uso de `array.array` exige tratar adequadamente exceções do tipo `TypeError` e `ValueError` quando dados fora do `typecode` são inseridos, convertendo-os para `TipoErro`.
2. **C++:** A destruição do ponteiro bruto (`delete[] dados`) e o bloqueio dos construtores de cópia e atribuição evitam *dangling pointers* e vazamento de memória.
3. **JavaScript:** O uso de `TypedArray` resolve o problema de arrays dinâmicos genéricos no JS, garantindo tipo básico de dados contíguo na memória.

User prompt: Gere as baterias de testes unitários e testes de estresse para as três implementações da classe Pilha.

Response: Abaixo estão as baterias de testes unitários e de estresse para as três linguagens. Os testes cobrem:

1. **Lógica e Exceções (Unitários):** Comportamento dos métodos, disparo correto de `PilhaCheiaErro`, `PilhaVaziaErro`, `TipoErro` e o funcionamento de `troca()`.
2. **Desempenho e Carga (Estresse):** Empilhamento e desempilhamento em massa (10^6 elementos) e verificação de consumo de tempo/memória.

1. Python (`python/test_pilha.py`)

Execução via `pytest` ou executando o arquivo diretamente com `python test_pilha.py`.

```

import time
import unittest
import array
from excecoes import PilhaCheiaErro, PilhaVaziaErro, TipoErro
from pilha import Pilha

class TestPilhaUnitario(unittest.TestCase):
    def test_operacoes_basicas(self):
        p = Pilha(capacidade=3, typecode='i')
        self.assertTrue(p.pilha_esta_vazia())

        p.empilha(10)
        p.empilha(20)
        self.assertEqual(p.tamanho(), 2)
        self.assertFalse(p.pilha_esta_vazia())

        self.assertEqual(p.desempilha(), 20)
        self.assertEqual(p.desempilha(), 10)
        self.assertTrue(p.pilha_esta_vazia())

    def test_excecao_pilha_cheia(self):
        p = Pilha(capacidade=2, typecode='i')
        p.empilha(1)
        p.empilha(2)
        self.assertTrue(p.pilha_esta_cheia())
        with self.assertRaises(PilhaCheiaErro):
            p.empilha(3)

```

```

def test_excecao_pilha_vazia(self):
    p = Pilha(capacidade=2, typecode='i')
    with self.assertRaises(PilhaVaziaErro):
        p.desempilha()

def test_excecao_tipo_incompativel(self):
    p = Pilha(capacidade=5, typecode='i') # Apenas inteiros
    with self.assertRaises(TipoErro):
        p.empilha("texto_invalido")

def test_metodo_troca(self):
    p = Pilha(capacidade=5, typecode='i')
    p.empilha(100)
    p.empilha(200)
    p.troca()
    self.assertEqual(p.desempilha(), 100)
    self.assertEqual(p.desempilha(), 200)

def test_troca_com_poucos_elementos(self):
    p = Pilha(capacidade=5, typecode='i')
    p.empilha(1)
    with self.assertRaises(PilhaVaziaErro):
        p.troca()

class TestPilhaEstresse(unittest.TestCase):
    def test_estresse_1m_elementos(self):
        n = 1_000_000
        p = Pilha(capacidade=n, typecode='i')

        inicio = time.time()
        for i in range(n):
            p.empilha(i)
        tempo_empilhar = time.time() - inicio

        self.assertTrue(p.pilha_esta_cheia())
        self.assertEqual(p.tamanho(), n)

        inicio = time.time()
        for i in range(n - 1, -1, -1):
            val = p.desempilha()
            assert val == i
        tempo_desempilhar = time.time() - inicio

        print(f"\n[Python Estresse - {n} elementos]")
        print(f"Tempo de Empilhar: {tempo_empilhar:.4f}s")
        print(f"Tempo de Desempilhar: {tempo_desempilhar:.4f}s")

if __name__ == "__main__":
    unittest.main()

```

2. C++ (cpp/main_test.cpp)

Compilação recomendada: g++ -O3 -std=c++11 main_test.cpp -o test_pilha && ./test_pilha

```

#include <iostream>
#include <cassert>
#include <chrono>
#include "Pilha.hpp"
#include "Excecoes.hpp"

void test_unitarios() {
    std::cout << "Executando testes unitarios (C++)... ";

    // Teste basico
    Pilha<int> p(3);
    assert(p.pilha_esta_vazia() == true);
    p.empilha(10);
    p.empilha(20);
    assert(p.tamanho() == 2);
    assert(p.desempilha() == 20);
}

```

```

// Teste Troca
p.empilha(30);
p.troca(); // topo tem 30 e 10. Apos troca: topo tem 10 e 30
assert(p.desempilha() == 10);
assert(p.desempilha() == 30);

// Teste Excecao PilhaCheiaErro
Pilha<int> p_cheia(1);
p_cheia.empilha(100);
try {
    p_cheia.empilha(200);
    assert(false); // Nao deve chegar aqui
} catch (const PilhaCheiaErro& e) {
    // Sucesso
}

// Teste Excecao PilhaVaziaErro
Pilha<int> p_vazia(2);
try {
    p_vazia.desempilha();
    assert(false);
} catch (const PilhaVaziaErro& e) {
    // Sucesso
}

std::cout << "OK!" << std::endl;
}

void test_estresse() {
    std::size_t n = 10000000; // 10 Milhões
    std::cout << "\n[C++ Estresse - " << n << " elementos]" << std::endl;

    Pilha<long long> p(n);

    auto t1 = std::chrono::high_resolution_clock::now();
    for (std::size_t i = 0; i < n; ++i) {
        p.empilha(static_cast<long long>(i));
    }
    auto t2 = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> dur_empilha = t2 - t1;

    auto t3 = std::chrono::high_resolution_clock::now();
    for (std::size_t i = 0; i < n; ++i) {
        p.desempilha();
    }
    auto t4 = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> dur_desempilha = t4 - t3;

    std::cout << "Tempo Empilhar: " << dur_empilha.count() << "s" << std::endl;
    std::cout << "Tempo Desempilhar: " << dur_desempilha.count() << "s" << std::endl;
}

int main() {
    test_unitarios();
    test_estresse();
    return 0;
}

```

3. JavaScript (js/pilha.test.js)

Pode ser executado com Node.js diretamente: `node pilha.test.js` ou integrado ao framework Jest.

```

const assert = require('assert');
const Pilha = require('./Pilha');
const { PilhaCheiaErro, PilhaVaziaErro, TipoErro } = require('./Excecoes');

function executarTestesUnitarios() {
    console.log("Executando testes unitários (JavaScript)...");

    // Testes básicos
    const p = new Pilha(3, Int32Array);
    assert.strictEqual(p.pilha_esta_vazia(), true);

    p.empilha(10);

```



```

p.empilha(20);
assert.strictEqual(p.tamanho(), 2);
assert.strictEqual(p.desempilha(), 20);

// Teste de Troca
p.empilha(30); // Estado atual: [10, 30]
p.troca();     // Estado atual: [30, 10]
assert.strictEqual(p.desempilha(), 10);
assert.strictEqual(p.desempilha(), 30);

// Teste PilhaCheiaErro
const pCheia = new Pilha(1);
pCheia.empilha(100);
assert.throws(() => pCheia.empilha(200), PilhaCheiaErro);

// Teste PilhaVaziaErro
const pVazia = new Pilha(1);
assert.throws(() => pVazia.desempilha(), PilhaVaziaErro);

// Teste TipoErro (Apenas numeros sao permitidos em TypedArray)
const pTipo = new Pilha(5);
assert.throws(() => pTipo.empilha("texto"), TipoErro);
assert.throws(() => pTipo.empilha(NaN), TipoErro);

console.log("OK!");
}

function executarTesteEstresse() {
  const n = 5_000_000; // 5 Milhões
  console.log(`\n[JS Estresse - ${n} elementos]`);
  const p = new Pilha(n, Float64Array);

  const inicioEmpilhar = performance.now();
  for (let i = 0; i < n; i++) {
    p.empilha(i);
  }
  const fimEmpilhar = performance.now();

  const inicioDesempilhar = performance.now();
  for (let i = 0; i < n; i++) {
    p.desempilha();
  }
  const fimDesempilhar = performance.now();

  console.log(`Tempo Empilhar: ${((fimEmpilhar - inicioEmpilhar) / 1000).toFixed(4)}s`);
  console.log(`Tempo Desempilhar: ${((fimDesempilhar - inicioDesempilhar) / 1000).toFixed(4)}s`);
}

executarTestesUnitarios();
executarTesteEstresse();

```

Dicas para a Seção de "Evidência de Testes" no POST_MORTEM.md:

- 1. Compare os tempos de execução:** Ao executar o teste de estresse, inclua no arquivo os dados de desempenho (tempo por operação e uso de memória).
- 2. Refatoração recomendada para o relatório:**
 - **Python:** Destaque a diferença de performance entre o uso do `array.array` da biblioteca padrão em relação a uma `list` nativa ingênua gerada por modelos de IA sem a devida diretiva de tipagem estrita.
 - **C++:** Relate a alocação prévia via bloco contíguo de memória (`new T[capacidade]`) comparada a alocações dinâmicas repetidas durante inserções.
 - **JS:** Apresente o uso de `TypedArray` (`Int32Array/Float64Array`) comparado aos arrays heterogêneos padrão da linguagem (`[]`).

User prompt: try: self._dados.append(dado) except (TypeError, ValueError) as err: raise TipoErro(f"O dado '{dado}' não é compatível com o tipo estipulado '{self._typecode}'.") from err o que isto faz?

Response: Esse trecho de código é responsável por **inserir um dado na pilha de forma segura**, garantindo que apenas o tipo correto de elemento seja aceito.

O que cada linha faz:

1. **try**: Abre um bloco para tentar executar um comando que pode falhar ou gerar erro.
2. **self._dados.append(dado)** Tenta adicionar o elemento (dado) no final do objeto `self._dados` (que é do tipo `array.array` da biblioteca padrão do Python).
 - Como o `array` do Python foi criado para guardar apenas um tipo específico (por exemplo, apenas números inteiros `'i'` ou decimais `'d'`), ele **rejeita automaticamente** qualquer dado incompatível.
3. **except (TypeError, ValueError) as err**: Captura a falha caso o `array.array` tente guardar um dado incompatível (por exemplo, se você tentar inserir uma string `"texto"` em uma pilha configurada para inteiros). O Python nativamente dispara um `TypeError` ou `ValueError` nessa situação.
4. **raise TipoErro(...) from err** Em vez de deixar o programa explodir com o erro genérico do Python, essa linha **intercepta a falha e lança a exceção personalizada TipoErro**, que atende rigorosamente ao requisito da especificação do trabalho da UFRJ. O sufixo `from err` mantém o histórico original do erro para facilitar o depurador (*debug*).

Exemplo prático de funcionamento:

```
# Criando uma pilha para armazenar inteiros ('i')
p = Pilha(capacidade=5, typecode='i')

p.empilha(10)      # OK! 10 é inteiro.
p.empilha("texto") # ERRO! A linha do try vai falhar e acionar o except,
                  # disparando: TipoErro: 0 dado 'texto' não é compatível...
```

User prompt: repare que faltou o arquivo `pilha.cpp`

Response: Análise perfeita! Como a classe `Pilha` em C++ foi projetada utilizando **Templates** (template `<typename T>`), a implementação inteira dos métodos costuma ficar no próprio arquivo de cabeçalho (`.hpp`). Em C++, o compilador precisa ter acesso a todo o código do modelo no momento da instanciação do template.

Contudo, para manter uma **estrutura de projeto modular clássica** com a separação em arquivo de cabeçalho (`.hpp`) e de implementação (`.cpp`), inclui-se o `.cpp` no final do `.hpp` ou realiza-se a instanciação explícita.

Abaixo está a separação correta em 3 arquivos para o diretório `cpp/`:

1. `cpp/Excecoes.hpp`

```
#ifndef EXCECOES_HPP
#define EXCECOES_HPP

#include <stdexcept>
#include <string>

class PilhaCheiaErro : public std::runtime_error {
public:
    explicit PilhaCheiaErro(const std::string& msg = "Pilha Cheia")
        : std::runtime_error(msg) {}
};

class PilhaVaziaErro : public std::runtime_error {
public:
    explicit PilhaVaziaErro(const std::string& msg = "Pilha Vazia")
        : std::runtime_error(msg) {}
};

class TipoErro : public std::invalid_argument {
public:
    explicit TipoErro(const std::string& msg = "Tipo Incompatível")
        : std::invalid_argument(msg) {}
};

#endif
```

2. `cpp/Pilha.hpp`

Contém as declarações da classe e faz a inclusão da implementação no final para viabilizar o uso de templates.

```
#ifndef PILHA_HPP
#define PILHA_HPP

#include <cstdint>
#include "Excecoes.hpp"
```

```

template <typename T>
class Pilha {
private:
    T* dados;
    std::size_t capacidade;
    std::size_t topo;

public:
    explicit Pilha(std::size_t cap);
    ~Pilha();

    // Desabilita cópia rasa para prevenir vazamento e double-free
    Pilha(const Pilha&) = delete;
    Pilha& operator=(const Pilha&) = delete;

    void empilha(T dado);
    T desempilha();
    bool pilha_esta_vazia() const;
    bool pilha_esta_cheia() const;
    void troca();
    std::size_t tamanho() const;
};

// Inclui a implementação das funções-membro baseadas em template
#include "Pilha.cpp"

#endif

```

3. cpp/Pilha.cpp

Contém as definições e corpo dos métodos da classe.

```

#ifndef PILHA_CPP
#define PILHA_CPP

#include "Pilha.hpp"

template <typename T>
Pilha<T>::Pilha(std::size_t cap) : capacidade(cap), topo(0) {
    if (cap == 0) {
        throw TipoErro("A capacidade deve ser maior que zero.");
    }
    dados = new T[capacidade];
}

template <typename T>
Pilha<T>::~~Pilha() {
    delete[] dados;
}

template <typename T>
void Pilha<T>::empilha(T dado) {
    if (pilha_esta_cheia()) {
        throw PilhaCheiaErro("A pilha atingiu a capacidade máxima.");
    }
    dados[topo++] = dado;
}

template <typename T>
T Pilha<T>::desempilha() {
    if (pilha_esta_vazia()) {
        throw PilhaVaziaErro("A pilha está vazia.");
    }
    return dados[--topo];
}

template <typename T>
bool Pilha<T>::pilha_esta_vazia() const {
    return topo == 0;
}

template <typename T>
bool Pilha<T>::pilha_esta_cheia() const {

```

```
        return topo == capacidade;
    }

    template <typename T>
    void Pilha<T>::troca() {
        if (topo < 2) {
            throw PilhaVaziaErro("A pilha necessita de ao menos dois elementos para efetuar a troca.");
        }
        T temp = dados[topo - 1];
        dados[topo - 1] = dados[topo - 2];
        dados[topo - 2] = temp;
    }

    template <typename T>
    std::size_t Pilha<T>::tamanho() const {
        return topo;
    }

#endif
```

Por que usar #include "Pilha.cpp" no final do .hpp?

- 1. **Separação de código:** Permite manter as assinaturas limpas no arquivo .hpp e a lógica de execução no arquivo .cpp.
- 2. **Compatibilidade com Templates:** Como a geração de código nativo do C++ para templates só ocorre quando o tipo (ex: int, float) é conhecido no momento de compilação, o compilador precisa enxergar o corpo dos métodos no mesmo instante em que lê o cabeçalho.
- 3. **Organização do repositório:** Garante que a estrutura de diretórios apresentada na árvore de arquivos contenha o arquivo Pilha.cpp sem ferir as regras do compilador C++.

User prompt: qual a diferença entre um arquivo cpp e hpp?

Response: A principal diferença está na **função de cada arquivo dentro do processo de compilação** da linguagem C++: o arquivo .hpp é o cabeçalho (*Header*) responsável por declarar a estrutura e a interface do código, enquanto o arquivo .cpp é a implementação onde reside a lógica executável das funções e métodos.

Característica	Arquivo Cabeçalho (.hpp)	Arquivo de Código (.cpp)
Papel Principal	Declaração: Define "o que existe" (interfaces, assinaturas, classes, estruturas, constantes).	Implementação: Define "como funciona" (o corpo das funções e a lógica dos métodos).
Processamento	Incluído por outros arquivos através de diretivas #include.	Compilado individualmente pelo compilador para gerar um arquivo objeto (.o ou .obj).
Conteúdo Comum	Assinaturas de métodos, definição de atributos de classes, escopos de namespaces, #define e declaração de exceções.	Instruções executáveis, algoritmos, alocação detalhada e lógica de controle.
Templates em C++	Costuma conter tanto a declaração quanto a implementação completa (devido à instanciação do compilador).	Usado para implementações não-template ou instanciações explícitas de tipos.

Exemplo Prático de Separação

1. Pilha.hpp (O Contrato / Interface)

```
#ifndef PILHA_HPP
#define PILHA_HPP

class Pilha {
private:
    int* dados;
    int topo;
    int capacidade;

public:
    Pilha(int cap);           // Apenas declaração do construtor
    ~Pilha();                 // Declaração do destruidor
    void empilha(int dado);   // Declaração do método
};

#endif
```

2. Pilha.cpp (A Execução / Lógica)

```
#include "Pilha.hpp"

Pilha::Pilha(int cap) : capacidade(cap), topo(0) {
    dados = new int[capacidade];
}

Pilha::~Pilha() {
    delete[] dados;
}

void Pilha::empilha(int dado) {
    dados[topo++] = dado;
}
```

Por que separar em dois arquivos?

1. **Organização e Ocultação de Detalhes:** Permite que outros programas ou módulos usem uma classe apenas incluindo o arquivo .hpp, sem ter acesso visual direto à complexidade da implementação contida no .cpp.
2. **Tempo de Compilação:** Se você alterar apenas a lógica interna dentro do arquivo .cpp, o compilador precisa recompilar apenas aquele arquivo específico, sem precisar recompilar todo o restante do projeto que depende do .hpp.
3. **Evitar Erros de Re-definição:** Se a lógica de uma função normal estivesse dentro de um .hpp e esse cabeçalho fosse incluído em múltiplos arquivos .cpp, o *linker* do C++ geraria um erro de definição duplicada (*multiple definition error*).

User prompt: o que é um template?

Response: Em C++ (e na programação em geral), um **template** (ou *gabarito/modelo*) é um recurso de **metaprogramação** que permite escrever código genérico, independente do tipo de dado.

Em vez de criar uma classe ou função específica para `int`, outra para `float` e outra para `char`, você escreve a estrutura **uma única vez** usando um "tipo fantasma" (geralmente chamado de `T`). O próprio compilador cria as versões específicas em tempo de compilação.

O Problema que o Template Resolve

Imagine que você precisa de uma classe `Pilha` de números inteiros (`int`). Depois, precisa de uma para números decimais (`double`). Sem templates, você teria que duplicar o código:

```
// Sem Template: Código Duplicado
class PilhaInt { int dados[100]; };
class PilhaDouble { double dados[100]; };
```

Se houver um bug ou melhoria, você precisa alterar manualmente cada uma das classes.

Como o Template Funciona

Com template, você define a classe usando a sintaxe `template <typename T>` (onde `T` representa qualquer tipo que será informado no futuro):

```
template <typename T>
class Pilha {
private:
    T* dados;           // Array do tipo T (pode ser int, double, char...)
    int topo;

public:
    void empilha(T dado) {
        dados[topo++] = dado;
    }

    T desempilha() {
        return dados[--topo];
    }
};
```

Uso na prática:

No momento em que você instancia o objeto, você passa o tipo desejado entre `< >`:

```
Pilha<int> pilhaDeInteiros;    // O compilador substitui T por 'int'
Pilha<double> pilhaDeFloats;   // O compilador substitui T por 'double'
Pilha<char> pilhaDeCaracteres; // O compilador substitui T por 'char'
```

O que Acontece nos Bastidores?

1. **Geração em Tempo de Compilação:** O compilador lê seu código Pilha<T>. Se você usar Pilha<int> e Pilha<float> no seu projeto, o compilador gera automaticamente na memória duas classes separadas e otimizadas para a sua máquina: uma para int e outra para float.
2. **Desempenho Sem Perda (Zero Overhead):** Diferente de linguagens dinâmicas (como Python), o uso de templates em C++ **não perde desempenho em tempo de execução**, pois a verificação de tipos e a criação do código ocorrem totalmente durante a compilação.

User prompt: `main_test` o que é este arquivo que foi gerado após a execução de `main_test.py`? o que eu vou precisar adicionar no `.gitignore`?

Response: O arquivo `main_test` (sem extensão) gerado na pasta do C++ é o **arquivo executável binário** resultante da compilação do seu código-fonte (`main_test.cpp`).

Quando você compila um programa em C++ utilizando um comando como:

```
g++ -O3 -std=c++11 main_test.cpp -o main_test
```

O compilador transforma o código em texto que você escreveu para **código de máquina** executável pelo seu sistema operacional.

Ele NÃO deve ser enviado ao GitHub. Binários executáveis mudam dependendo do sistema de quem compila (Linux, macOS, Windows) e ocupam espaço desnecessário no repositório.

O que você precisa adicionar no `.gitignore`?

Crie ou edite o arquivo `.gitignore` na raiz do seu repositório para evitar que arquivos compilados, caches e temporários gerados na execução das três linguagens (Python, C++ e JS) sejam rastreados pelo Git.

Adicione as seguintes regras:

```
# =====
# C / C++ (Compilados e Executáveis)
# =====
# Executáveis binários sem extensão
main_test
test_pilha
*.exe
*.out
*.app

# Arquivos objeto e bibliotecas geradas na compilação
*.o
*.obj
*.so
*.dylib
*.a

# =====
# Python (Caches e Arquivos Temporários)
# =====
__pycache__/
*.py[cod]
*$py.class
.pytest_cache/
.coverage
htmlcov/

# =====
# JavaScript / Node.js
# =====
node_modules/
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# =====
# Arquivos de IDEs e Sistema Operacional
# =====
.vscode/
.idea/
.DS_Store
Thumbs.db
```

O que fazer se o `main_test` já tiver sido enviado para o Git?

Se o executável já foi adicionado pelo Git (aparece em `git status`), remova-o do rastreamento executando no terminal:

```
git rm --cached cpp/main_test
git commit -m "chore: remove binario executavel do rastreamento e atualiza gitignore"
```

User prompt: como rodar javascript

Response: Para executar arquivos JavaScript, o método mais prático no contexto do seu projeto (fora do navegador) é utilizando o **Node.js** diretamente no terminal.

1. Pré-requisito: Verificar o Node.js

Abra o terminal ou o *Prompt de Comando* e verifique se o Node.js está instalado:

```
node -v
```

Se retornar a versão (ex: v18.16.0 ou v20.x.x), o ambiente está pronto. Se não reconhecer o comando, baixe e instale a versão LTS no site oficial do [Node.js](https://nodejs.org/).

2. Rodando um Arquivo JavaScript pelo Terminal

Navegue até a pasta onde está o seu arquivo .js e execute o comando node:

```
# Entrar na pasta do projeto JavaScript
cd js
```

```
# Executar o arquivo desejado
node Pilha.js
```

3. Rodando os Testes do Projeto (pilha.test.js)

Para rodar a bateria de testes unitários e de estresse que criamos para a classe Pilha:

```
node pilha.test.js
```

O Node.js executará o script e exibirá no terminal o resultado dos testes básicos e os tempos de desempenho do teste de carga:

```
Executando testes unitários (JavaScript)...
OK!
```

```
[JS Estresse - 5000000 elementos]
Tempo Empilhar: 0.1245s
Tempo Desempilhar: 0.0812s
```

4. Alternativa: Modos Interativos e Navegador

- **REPL do Node (Terminal Interativo):** Digite apenas node e aperte Enter para abrir um terminal onde você pode digitar código JS linha por linha.
 - **Navegador Web:** Você também pode rodar JS criando um arquivo index.html e vinculando o script via `<script src="Pilha.js"></script>`, ou inspecionando qualquer página com F12 e usando a aba **Console**.
-